

# Использование языка программирования Julia для решения математических задач

## Часть I

### **Авторы:**

#### **Чуракова Юлия Романовна**

*Студентка факультета математики и компьютерных наук  
Кубанского Государственного Университета*

#### **Алексеев Евгений Ростиславович**

*Кандидат технических наук, доцент  
Доцент кафедры информационных технологий  
Кубанского Государственного Университета*

---

**Полная версия пособия доступна в репозитории:**

<https://github.com/JuliaChurakova/Julia>

---

# Литература

1. Энгхейм, Э. Julia в качестве второго языка / Э. Энгхейм. – Москва: ДМК Пресс, 2023. – 446 с.
2. The Julia Programming Language: сайт. – URL: <https://julialang.org> (дата обращения: 10.10.2025).
3. Белов, Г. В. Краткое описание языка программирования Julia и некоторые примеры его использования/ Г. В. Белов. – Москва: МГТУ им. Н.Э. Баумана, 2024. – 108 с.
4. Шеррингтон, М. Осваиваем язык Julia / М. Шеррингтон. – Москва: ДМК Пресс, 2017. – 416 с.
5. Антонюк, В. А. Язык Julia как инструмент исследователя / В. А. Антонюк. – Москва: Физический факультет МГУ им. М. В. Ломоносова, 2019. – 48 с.

## Глава 7 Макросы

Макросы в Julia похожи на функции, но есть несколько отличий. Они определяются с помощью ключевого слова **macro** (вместо **function**) и вызываются с символом **@** перед именем макроса. В отличие от функций, для вызова макроса не обязательно использовать скобки – достаточно указать параметры через пробел.

```
@some_macro arg1 arg2
```

Макросы выполняются на этапе компиляции **до того, как код начнёт выполняться**. То есть, они изменяют сам код, а не его результаты. Когда вы вызываете макрос, он преобразует код в абстрактное синтаксическое дерево (AST), и это позволяет манипулировать выражениями и структурой кода.

Важно, что макросы работают не с конкретными значениями переменных, а с **выражениями** (то есть, с самими кусками кода). Это позволяет создавать программы, которые могут изменять или генерировать код во время работы. Это называется **метапрограммированием**.

### 7.1 Встроенные макросы

Макрос **@time** измеряет время выполнения кода и сообщает о затраченном времени, а также о расходе памяти.

```
In [12]: @time begin
           x = 0
           for i in 1:1000000
               x += i
           end
       end
```

```
end  
end
```

0.071738 seconds (2.00 M allocations: 30.509 MiB, 32.57% gc time)

Макрос **@elapsed** – это упрощённая версия **@time**. Возвращает время выполнения выражения, но не выводит его в консоль.

```
In [14]: t = @elapsed begin  
          x = 0  
          for i in 1:1000000  
              x += i  
          end  
        end  
println("Время выполнения: ", t)
```

Время выполнения: 0.04051345

Макрос **@which** используется для того, чтобы узнать, какой метод будет вызван для конкретного выражения.

```
In [15]: @which println("Hello, world!")
```

Out[15]: println(xs...) in Base at [coreio.jl:4](#)

Макрос **@show** используется для вывода значений выражений на экран. Это полезно для отладки, чтобы видеть результаты вычислений прямо в процессе выполнения.

```
In [20]: x = 42  
         @show x
```

x = 42

Out[20]: 42

Макрос **@assert** используется для проверки условий. Если условие ложно, макрос генерирует ошибку и выводит проблемное выражение.

```
In [21]: x = 5  
         @assert x > 0    # Условие истинно, ошибок не будет  
  
         @assert x < 0    # Ошибка
```

AssertionError: x < 0

Stacktrace:

```
[1] top-level scope  
  @ In[21]:4
```

## 7.2 Создание макросов

Теперь, когда мы рассмотрели основные встроенные макросы, давайте перейдем к созданию собственных макросов.

Синтаксис для создания макроса выглядит так:

```
macro имя_макроса(параметры)
  тело_макроса
end
```

```
In [24]: macro hello(name)
         return :("Привет, " * $name)
       end
```

```
Out[24]: @hello (macro with 1 method)
```

```
In [25]: @hello("Мир")
```

```
Out[25]: "Привет, Мир"
```

Макросы возвращают код в виде выражений, используя символ `:`. Для вставки значений переменных в код макроса используется символ `$`.

```
In [26]: macro add(a, b)
         println("Вычисляем сумму: ", a, " + ", b)
         return :($a + $b)
       end
```

```
Out[26]: @add (macro with 1 method)
```

```
In [27]: @add 3 5
```

```
Вычисляем сумму: 3 + 5
```

```
Out[27]: 8
```